

Netcompany



C0200 - User Guide – Feniks AI

Version	1.5
Status	05 - Approved
Approver	Peter Troelsen Rosenberg Heltoft
Author	Daniel Cheveyo

© Copyright 2026 Netcompany
All rights reserved.

Document history

Version	Date	Author	Status	Comments
0.1	10-06-2026	Daniel Cheveyo	Completed	
1.0	08-07-2026	Peter Heltoft	Approved	Approved for internal 0.3 roll-out.
1.1	09-07-2026	Peter Heltoft	Approved	Feedback after v0.3 rollout.
1.2	13-07-2026	Filip Liwiński	Approved	Troubleshooting chapter moved to FAQ in NCAI Toolkit.
1.3	06-08-2026	Daniel Cheveyo	Completed	Updating guidelines with functionality and user-interface supported in Feniks AI 1.0.0
1.3	10-08-2026	Peter Troelsen Rosenberg Heltoft	Approved	
1.4	11-08-2026	Daniel Cheveyo	Completed	Updated section “3.2.3 Setting Up MCP Servers” that remote MCP servers are not prohibited, but not recommended and that Trust Centre mitigates the risk.
1.5	04-09-2026	Christina Brix Christensen	Completed	Added a new section 2.1.1: enabling Hyper V, 2.1.3: installation guide for Rancher desktop. Updated section 2.1.2 with more steps for WSL installation and a note to 2.6.1.1: app settings for a smoother startup
1.5	08-09-2026	Daniel Cheveyo	Approved	

References

Reference	Title	Author
[C0200 – Agentic AI Guidelines]	C0200 - User Guide - Agentic AI Guidelines	Bjarne Kock
[C0200 – Jira Confluence MCP Setup]	C0200 - User Guide - Jira Confluence MCP Setup	Troels Madsen
[C0200 – Toolkit MCP Setup]	C0200 - User Guide - Toolkit MCP Setup	Troels Madsen
[C0200 – Feniks Test]	C0200 - User Guide - Feniks Test	Bjarne Kock

Table of contents

1	Introduction.....	4
1.1	Reading Guidelines	4
1.2	Audience	4
1.3	Purpose	4
2	Feniks AI Control Application	5
2.1	System Requirements	5
2.1.1	Enabling Hyper V	5
2.1.2	WSL Installation	6
2.1.3	Container runtime engine installation (Rancher Desktop)	6
2.2	Installation and First-Time Setup	6
2.2.1	Installing the Application.....	7
2.3	Overview Dashboard	13
2.3.1	Container Status	14
2.3.2	Component Cards	14
2.4	Managing Components	15
2.4.1	LLM-Bridge.....	15
2.4.2	MCP-Gateway.....	16
2.5	Skills.....	18
2.5.1	Workspaces	18
2.5.2	Installed Skills	19
2.5.3	Skill Library.....	19
2.6	Settings	21
2.6.1	General	21
2.6.2	OpenCode For Feniks	23
2.6.3	Notifications.....	23
2.6.4	Software Updates.....	24
2.6.5	About.....	25
2.7	User Menu	25
2.8	Troubleshooting.....	25
3	OpenCode For Feniks Agentic AI Assistant.....	25
3.1	Working with OpenCode For Feniks.....	26
3.1.1	The OpenCode For Feniks Interface	26
3.1.2	Working with Agents.....	31
3.2	Using OpenCode For Feniks in your project.....	33
3.2.1	Adding Custom Instruction Files.....	33
3.2.2	Embedding File-Read Directives.....	34
3.2.3	Setting Up MCP Servers	34
3.2.4	PostgreSQL MCP Pro	36
3.2.5	Toolkit MCP.....	37
3.2.6	Configuring File Exclusions	37
3.3	Getting Help	38
3.3.1	Debugging Tools	38
3.3.2	Resources	39
3.4	Known pitfalls and problems.....	39
3.4.1	Looping behaviour in plan mode	39
3.4.2	Burning through daily token allowance	40

1 Introduction

Feniks AI is Netcompany's framework for AI-accelerated enterprise software delivery, designed to combine the speed of agentic AI with the control required for complex and society-critical systems.

Feniks AI is designed to ensure that the benefits of agentic AI are realised within a controlled, predictable, and sovereign delivery model – particularly in sectors where reliability and compliance are critical.

This guide presents how Feniks AI and its components can be installed, configured and used.

Installation and usage of Feniks Test is optionally done via Feniks AI, see section 2.2.1.3 and [C0200 – Feniks Test].

1.1 Reading Guidelines

This document is split into two parts serving different audiences:

1. **Feniks AI** is for any user who has received the Feniks AI executable (.exe). No source code or developer tooling is required. It covers installation, first-time setup, and all actions available through the graphical interface: starting and stopping components, managing skills, configuring API keys, and checking for updates.
2. **OpenCode For Feniks** is an agentic tool for engineers and consultants who use OpenCode for project work. OpenCode may be installed automatically as part of the Feniks setup wizard, see section 0. All existing OpenCode guidance applies regardless of how it was installed.

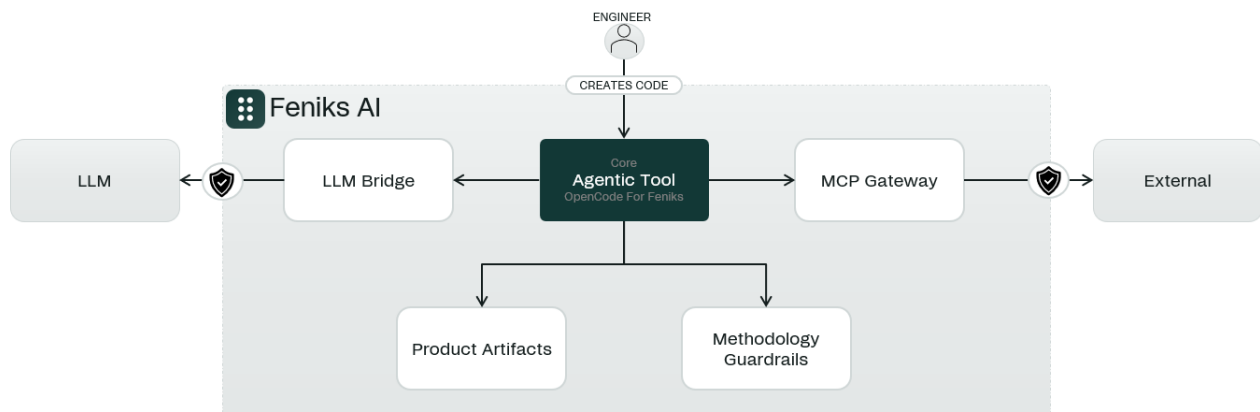


Figure 1: Feniks AI component overview

1.2 Audience

This guide is intended for multiple groups of users who will use Feniks AI as part of their project work:

- Netcompany architects, engineers, consultants and business consultants
- External users and parties

1.3 Purpose

The purpose of this document is to describe how development can be accelerated using Feniks AI, and how to use Feniks AI in the most effective way.

2 Feniks AI Control Application

Feniks AI is a desktop control center for the Netcompany AI ecosystem. It manages AI infrastructure components — the LLM Bridge and MCP Gateway — as containers on your machine. It also installs and configures AI coding tools (called harnesses): OpenCode for Feniks.

As an external user to Netcompany with the Feniks executable, everything you need to do is available through the graphical interface. You do not need access to source code, a terminal, or any developer tooling beyond the prerequisites below.

2.1 System Requirements

Requirement	Details
Operating system	Windows 10/11 (x64)
WSL	with kernel 5.15 or newer, see section 2.1.1
Container runtime engine	One of the following must be installed and running: 1. Rancher Desktop: Version 1.22.3 or later – Choose dockerd container engine 2. Podman Desktop: Version 1.29.1 or later 3. Docker Desktop: Version 4.85.0 or later (requires separate license)
Disk space	At least 4 GB free for container images
Feniks API key	Must be obtained, see section 2.2.1.2

Table 1: Feniks AI system requirements

Important: One of the listed container runtime engines must be running before you launch Feniks. If Docker or Podman is not running, Feniks will detect this and prompt you to start it before proceeding.

It is important that you follow this order for installation to ensure a successful installation of Feniks AI.

2.1.1 Enabling Hyper V

Before you start any of your installations, you need to ensure that Hyper V is enabled. To verify if this is already enabled run the following powershell command with elevated trust (administrator mode):

1. Request admin privileges via Heimdal
2. Open Windows Powershell as administrator
3. Run this command and press enter: `Get-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V`
4. Check the output of this command. Does it say “enabled” or “disabled”

```

FeatureName      : Microsoft-Hyper-V
DisplayName      : Hyper-V Platform
Description      : Provides the services that you can use to create and manage virtual machines and their resources.
RestartRequired : Possible
State            : Enabled
CustomProperties :
  
```

5. If state is “disabled”, run this command and press enter: `Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V -All`

6. It will now ask you to restart the PC. Choose “y” (for yes) and press enter.

2.1.2 WSL Installation

Disregarding what container runtime engine is chosen, it is required to have WSL installed before they are installed.

1. Request admin privileges via Heimdal
2. Open Command Prompt as administrator
3. Run this command and press enter: `wsl --install`
 - a. If this doesn't work, run this command instead: `wsl --install --web-download`
4. Restart the computer
5. Repeat step 1-2
6. To check if the Kernel version is 5.15 or newer, run this command and press enter: `wsl --version`

```
C:\Users\dch>wsl --version
WSL version: 2.7.11.0
Kernel version: 6.18.33.2-2
```

2.1.3 Container runtime engine installation (Rancher Desktop)

Note: *NBS employees are only allowed to install Rancher Desktop*

1. Go to Company Portal and install Rancher Desktop and open when installation is complete.
2. Choose “dockered (moby)” in pop-up and wait until it is done loading (bottom right corner)
 - a. If you get a new pop-up when done loading from Microsoft security about access to host-switch, you can press cancel
3. Check status OK and running (No error messages (check under “containers”))
 - a. If error message *Error response from daemon: failed to connect to the backend: timed out dialing Hyper-V socket* is appearing under the “Container” tab, it could be because you have installed Feniks AI before Rancher was up and running correctly. Here is how to fix it:
 - i. Reinstall both Rancher and Feniks AI – don't open them
 - ii. Make sure no instances of either are running
 - iii. Run PowerShell as admin
 - iv. Run following 3 commands:
 1. `wsl --shutdown`
 2. `Restart-Service vmcompute`
 3. `Restart-Service vmms`
 - v. Open Rancher Desktop
 - vi. Check status Ok and running (No error messages (check under “containers”))
 - vii. Go to “Preferences” à behavior
 - viii. Check the box “automatically start at login” and “start in the background” à apply

2.2 Installation and First-Time Setup

This section presents a step-by-step guide on how to install Feniks AI for both NC and external users.

2.2.1 Installing the Application

1. Download the installer from <https://feniks-ai.netcompany.com/downloads>. Please note that Netcompany users can alternatively download and install Feniks AI via Company portal instead. The following installers are available:
 - a. Windows: .exe (NSIS installer)
2. Run the installer and follow the on-screen prompts. No administrator rights are required for a per-user installation.
3. After installation, Feniks appears in the Start Menu as Feniks AI.

Note: Feniks minimises to the system tray on close. The first time this happens, a notification will inform you that Feniks is still running in the menu bar. Click the tray icon to reopen the window at any time.

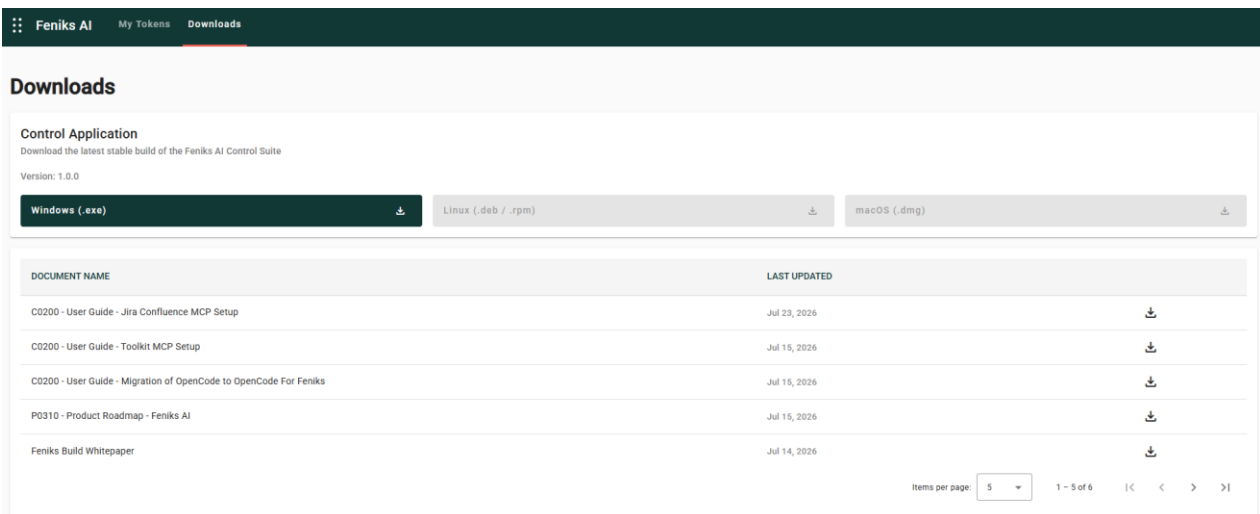


Figure 2: Feniks AI download installer page

2.2.1.1 Screen 1 — Update Check

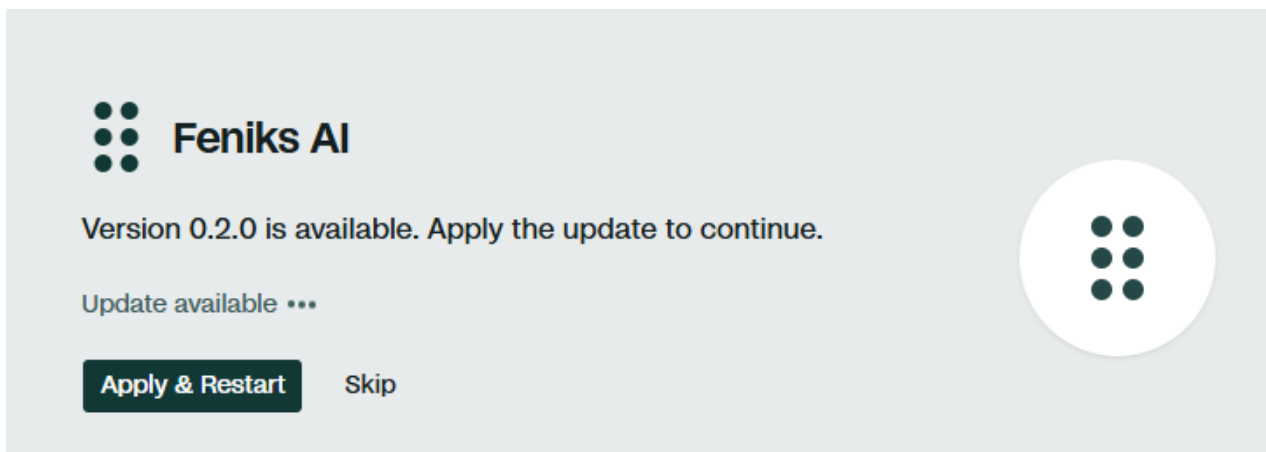


Figure 3: Feniks AI installation update check

Feniks automatically checks for a newer version of itself. The screen shows one of the following states:

State	Meaning
Checking for app updates...	The update check is in progress.

Downloading the latest version...	A newer version was found and is being downloaded.
Installing update...	The update is being applied.
You're up to date	No update needed. Feniks AI will proceed when button Continue is pressed.
We couldn't check for updates	The update server could not be reached.

Table 2: First-time update states

If the update check fails, click **Retry** to try again, or **Skip** to proceed without updating. Feniks advances to the next screen automatically once the check completes successfully.

2.2.1.2 Screen 2 — Feniks API Key

Feniks AI asks for your Feniks AI API key. This key authenticates you with Feniks AI services and allows the LLM Bridge to access AI models.

If a key is already saved from a previous installation, or if the key is detected from an environment variable, this screen confirms it and lets you proceed immediately.

To enter the key in Feniks AI:

1. Paste the key into the **API Key** field. See sections 2.2.1.2.1 and 2.2.1.2.2 for how the key can be obtained:

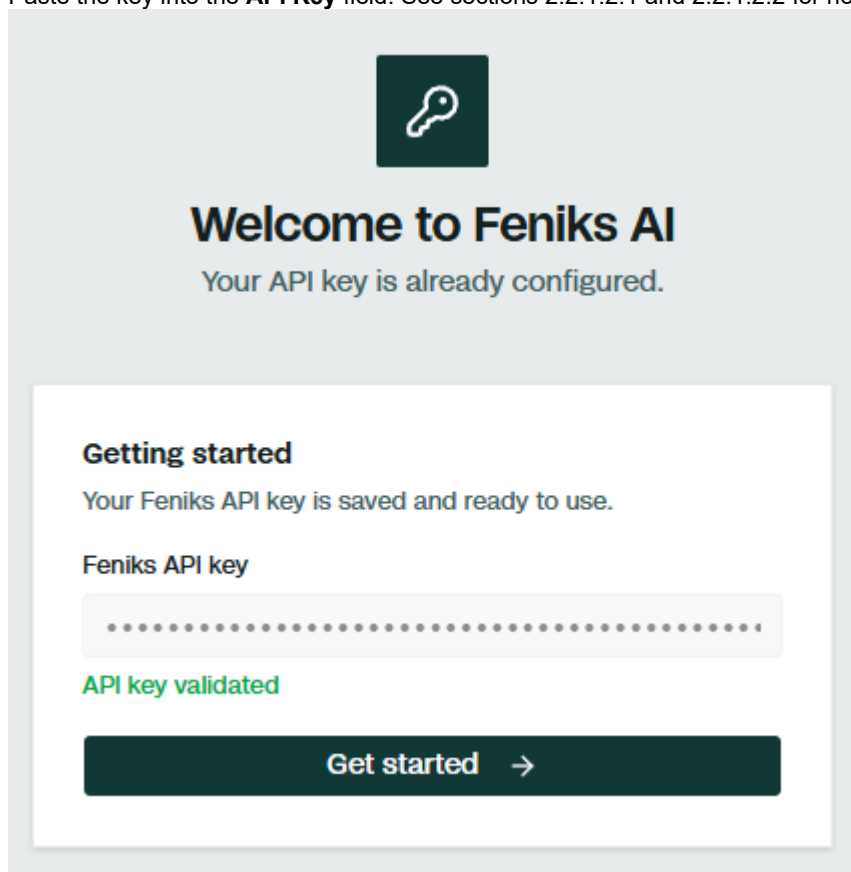


Figure 4: Feniks AI installation Feniks API key

2. Click **Get started**. Feniks validates the key before proceeding.

If validation fails, check that you copied the full key without extra spaces and try again.

2.2.1.2.1 Netcompany Users

To obtain a Feniks API key do the following steps:

1. Visit: <https://selfservice.netcompany.com/cortex/my-cortex-token>

My Cortex Token

Request an API token to integrate with Cortex AI services. Usage will be billed to the selected delivery.

Delivery *

Purpose *

Describe the AI usage for this project...

From Date: 17/03/2026 To Date: dd/mm/yyyy *

The following approver will be notified:

Approver

Change

Comments

Add a comment

Submit Request

Figure 5: Netcompany self-service requested Feniks API token

2. Enter your project or product/platform delivery name. Add specific details under “Purpose” for approver if needed.
3. Set the approver to your AI lead or lead Architect.
4. Once approved, click "Copy to Clipboard" to copy your API key.

My Cortex Token

Status: **APPROVED**

Delivery: NCDK0021 - Unpaid vacation

Valid Period: 17/03/2026 - 29/04/2026

Purpose: Test

API Token: Copy to Clipboard

Use this token to authenticate your requests to Cortex. Usage is billed to the project above.

Comments

Piotr Kominek
Test

Add a comment

Figure 6: Netcompany self-service approved Feniks API token

2.2.1.2.2 External Users

To obtain a Feniks API key do the following steps:

1. Visit: <https://feniks-ai.netcompany.com/my-tokens>

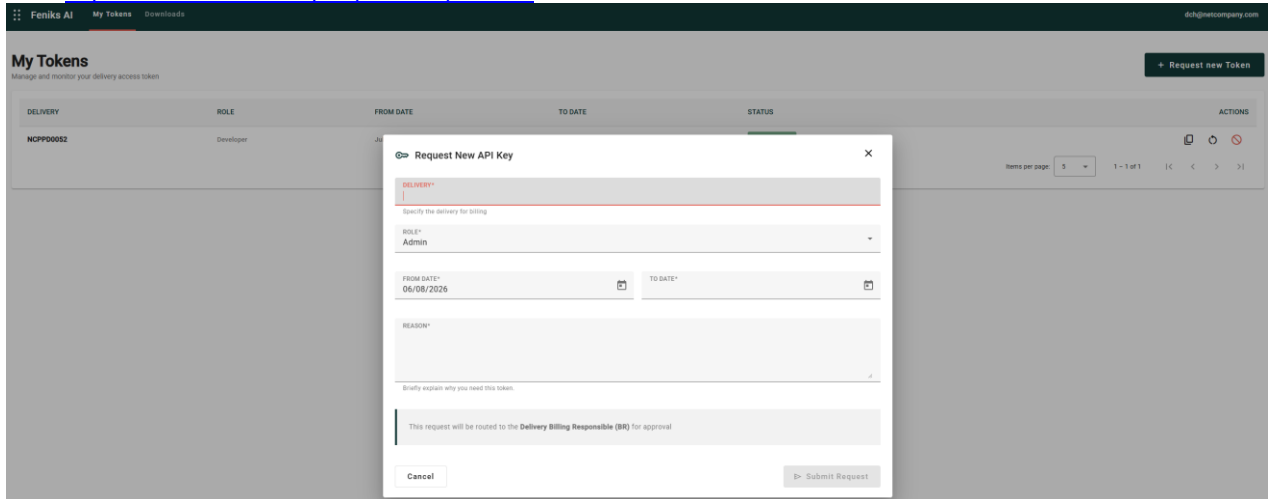


Figure 7: Feniks AI requested Feniks API token

2. Enter your project or product/platform delivery name and desired role. Add specific details under “Purpose” for approver if needed.
3. Once approved by delivery billing responsible, click "Copy to Clipboard" to copy your API key.

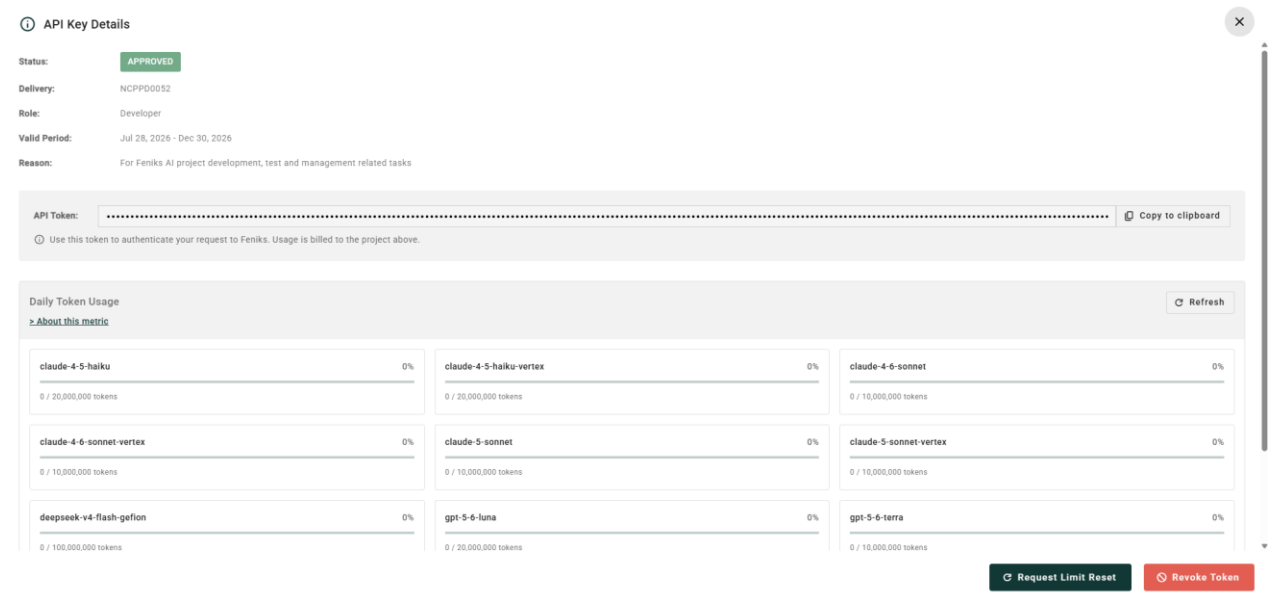


Figure 8: Feniks AI approved Feniks API token

2.2.1.3 Screen 3 — Installing Components

Feniks detects whether Docker, Rancher or Podman is running. If neither Docker, Rancher nor Podman are found:

1. Open Podman/Docker/Rancher Desktop from the Start Menu.
2. Wait until the engine is fully started (the tray icon stops animating).
3. Click **Retry**.
4. If this doesn't solve the problem and you can see an error message in either Feniks AI or Rancher Desktop:
Error response from daemon: failed to connect to the backend: timed out dialing Hyper-V socket - go back to section 2.1.3 and follow the steps for this error message.

Once Docker, Rancher or Podman is running, Feniks downloads the container images and starts them. A live log panel shows progress. The installation proceeds in phases:

- **Phase 1:** Mandatory containers are downloaded and started.
- **OpenCode Install:** OpenCode for Feniks is installed as a native component.
- **OpenCode Config Sync:** Default configuration is synced to `~/.config/opencode/opencode.json`.
- **Phase 2:** Optional components are installed based on your selection, see figure below and read the following guidelines **[C0200 – Jira Confluence MCP Setup]** and **[C0200 – Toolkit MCP Setup]**. It is recommended to skip this step for first time installation as they can be installed afterwards, see section 2.4.2.3.
(Note: **NBS employees** are not allowed to connect to the NC MCP's – Toolkit is the only exception)

Setting up

This may take a few minutes.

- ✓ OpenCode for Feniks
- ✓ OpenCode Configuration
- Additional Components

✓ Atlassian MCP server Installed

Atlassian MCP server

JIRA_PERSONAL_TOKEN Set

Enter JIRA_PERSONAL_TOKEN



CONFLUENCE_PERSONAL_TOKEN Set

Enter CONFLUENCE_PERSONAL_TO



✓ SonarQube MCP server Installed

SonarQube MCP server

SONARQUBE_TOKEN Set

Enter SONARQUBE_TOKEN



✓ Toolkit MCP server Installed

Toolkit MCP server

GOTO_URL Set

Enter GOTO_URL



GOTO_WRITE_ALLOWED_LISTS Set

Enter GOTO_WRITE_ALLOWED_LISTS



GOTO_COOKIE_HOST_DIR Set

Enter GOTO_COOKIE_HOST_DIR



Install Selected (3) →

Skip remaining

Figure 9: Figure 7: Feniks AI installation additional components installation

The label cycles through:

- Downloading components...
- Loading container images...
- Starting containers...
- Components running — installation complete.

This may take several minutes on first run. Do not close the application during this step. If the step fails, an error is shown with a **Retry** button.

The **OpenCode Configuration** may ask to **Overwrite** the existing configurations, in case this happens it is recommended to allow the configurations to be overwritten automatically with a default configuration file.

Once all three steps are complete, click **Open Feniks AI** to enter the application. If you want to continue without completing setup, click **Skip for now** or **Skip remaining**.

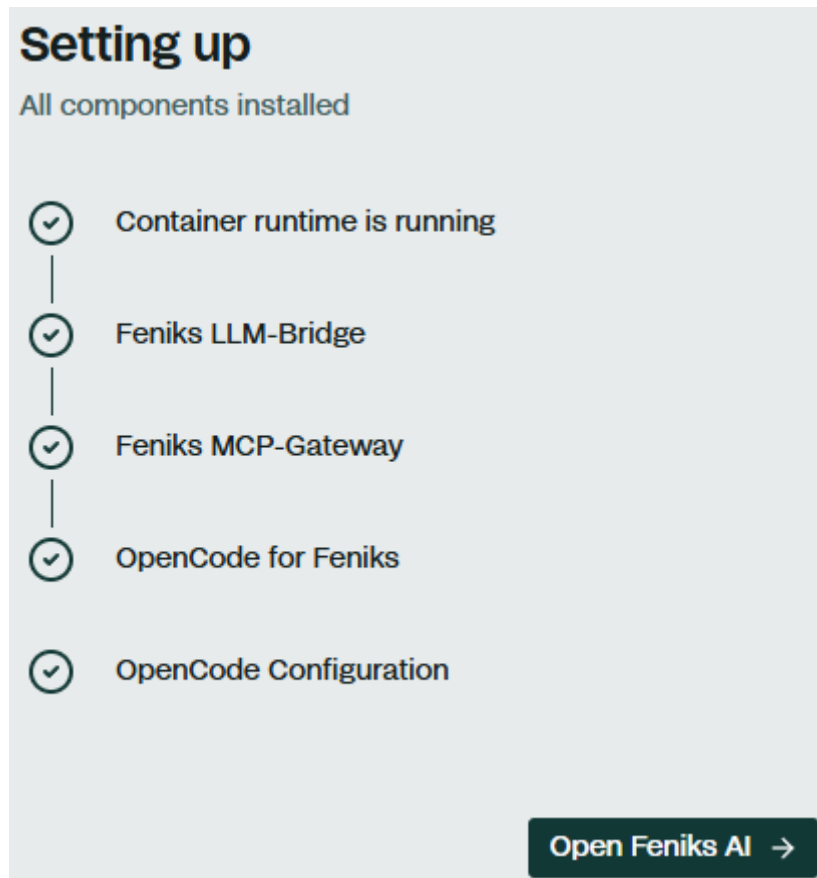


Figure 10: Feniks AI installation component installation completed

You can re-run setup at any time from the user menu (your initials in the top-right corner) → **Run Setup**.

2.3 Overview Dashboard

The **Overview** dashboard is the first screen you see after setup. It gives a live overview of the two Feniks AI core components:

- LLM-bridge

- MCP-gateway
- OpenCode for Feniks, which starts a separate graphical install wizard to reinstall OpenCode For Feniks.
- Feniks Test, which starts a separate graphical install wizard. Please refer to [C0200 – Feniks Test].
- Toolkit Login, which starts a separate graphical install wizard. Please refer to [C0200 – Toolkit MCP Setup].

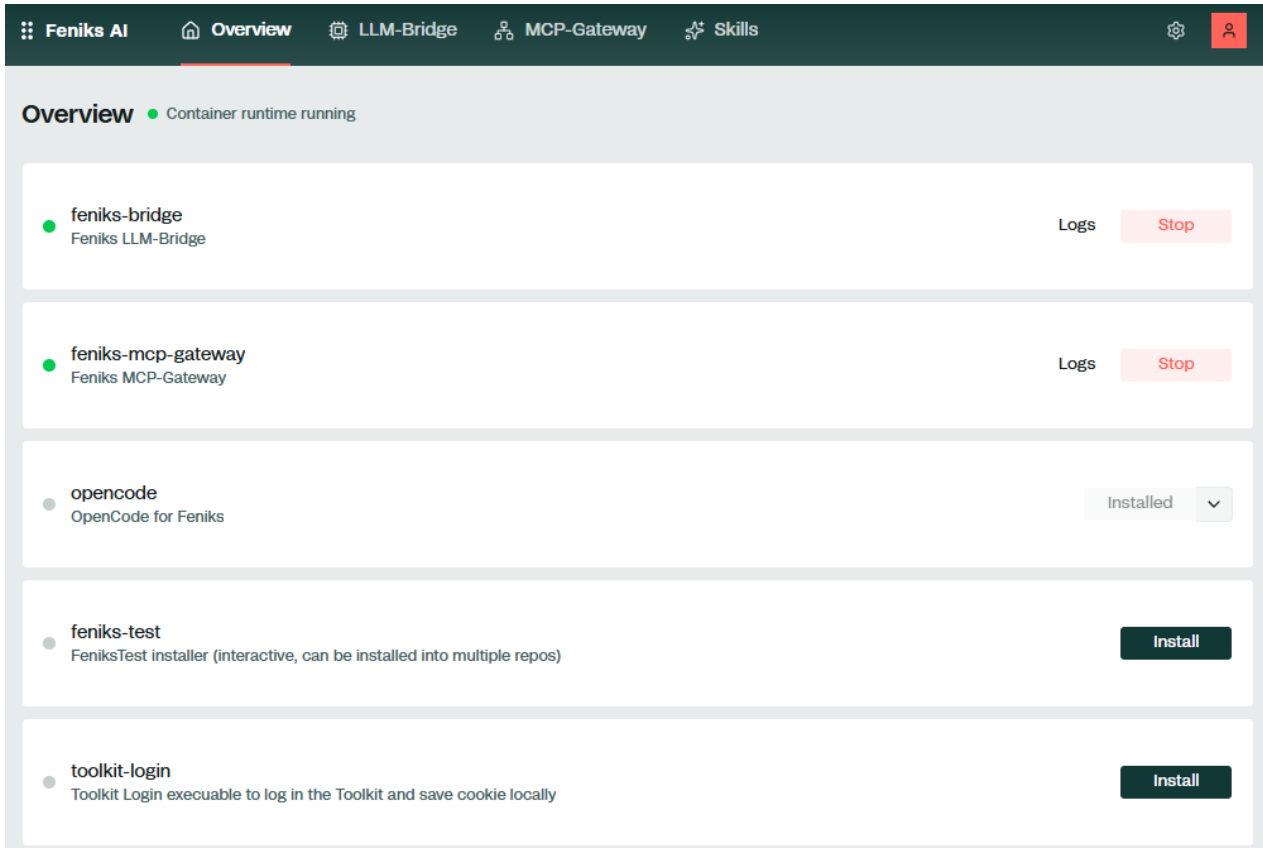


Figure 11: Feniks AI overview dashboard

2.3.1 Container Status

A small indicator at the top of the Dashboard shows whether containers are running:

- ● Green dot — Container is running: Components can be started and stopped normally.
- ● Red dot — Container is unavailable: Components cannot run. Start the container runtime engine and wait for Feniks AI to detect it automatically (it checks every 5 seconds).
- ● Gray dot — Not depending on containers.

2.3.2 Component Cards

Each component is shown as a card with:

- Name and description of the component.
- Status indicator — see table below.
- Logs button — opens a live log panel for the component (only visible when the container exists).
- Action button — starts, stops, or installs the component.

Status	Meaning
Running	The container is active and healthy.
Starting	The container is initialising. Wait a few seconds.
Stopped	The container exists but is not running. Click Start to resume.
Unhealthy	The container is running but failing health checks. Check logs.
Not installed	The image has not been pulled yet. Click Install & start.

Table 3: Feniks AI component status

2.4 Managing Components

Feniks AI manages two core components to governance the agentic AI harness with control and sovereignty:

- LLM Bridge
- MCP Gateway

Each component has its own dedicated tab in Feniks AI and can be configured to fit a project’s needs.

2.4.1 LLM-Bridge

The LLM-Bridge is an OpenAI-compatible LLM proxy with a synchronous middleware pipeline and an asynchronous processing queue.

The LLM-Bridge sits between OpenCode For Feniks and the LLM. Every request pass through a configurable set of modules before being forwarded; a copy is simultaneously queued for offline processing — without adding latency to the response.

When navigating to the **LLM-Bridge** in the top menu. The application will be shown as illustrated in the figure below:

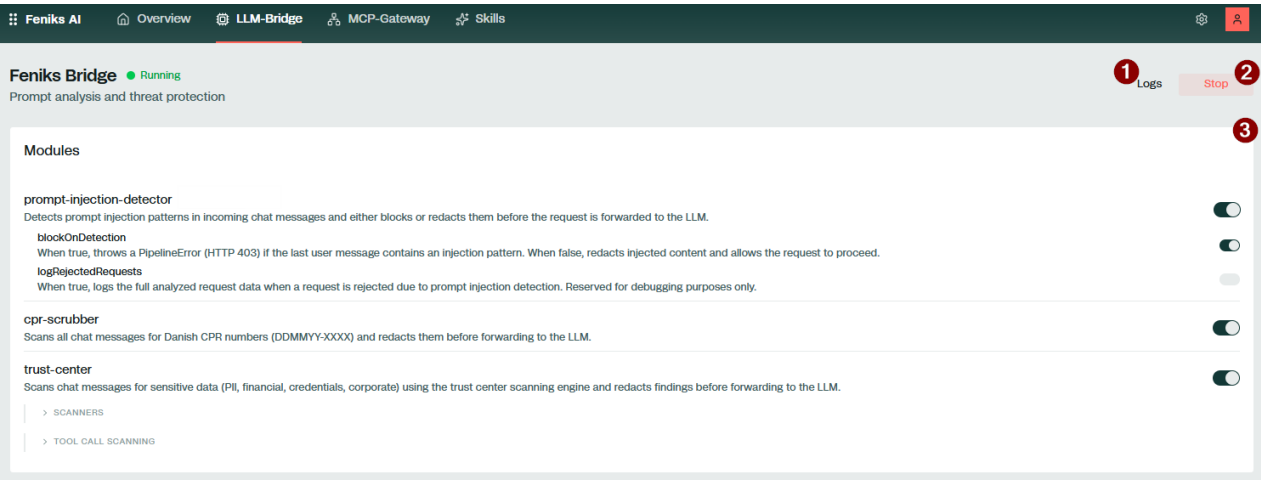


Figure 12: Feniks AI LLM-bridge

Point number in figure	Related subsection
------------------------	--------------------

1	2.4.1.1
2	2.4.1.2
3	2.4.1.3

Table 4: Feniks AI LLM-bridge legend

The LLM Bridge routes requests from your AI coding tool to the configured LLM models. It exposes port **4666** on your local machine.

2.4.1.1 Viewing Logs

Click **Logs** to open the live log panel. Logs stream in real time and are useful for diagnosing connection errors or unexpected behaviour.

2.4.1.2 Starting and Stopping

- Click **Install & start** if the Bridge has not been installed yet.
- Click **Start** if the Bridge is stopped.
- Click **Stop** (requires confirmation) to stop the running Bridge.

2.4.1.3 Modules

When the Bridge is running, the Modules section lists the available LLM integrations. Each module has:

- Name and description of the integration.
- Mandatory badge — mandatory modules cannot be disabled.
- Toggle switch — enable or disable optional modules individually.

Changes to module toggles take effect immediately without restarting the Bridge.

If no modules are listed, the Bridge may still be starting up. Wait a few seconds and the list will populate.

2.4.2 MCP-Gateway

MCP (Model Context Protocol) Gateway is an extensible, role-oriented gateway that combines MCP tool lists from multiple MCP servers, called providers, and forwards tool calls through a single-entry point.

When navigating to the **MCP-Gateway** in the top menu. The application will be shown as illustrated in the figure below:

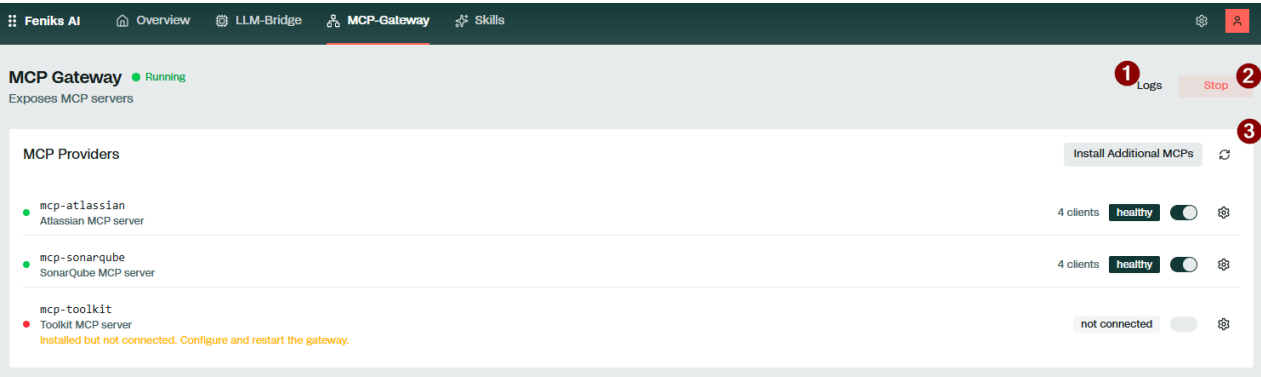


Figure 13: Feniks AI MCP-Gateway

Point number in figure	Related subsection
1	2.4.2.1
2	2.4.2.2
3	2.4.2.3

Table 5: Feniks AI MCP-gateway legend

The MCP Gateway exposes MCP servers to connected AI clients. It exposes port 4667 on your local machine.

2.4.2.1 Viewing Logs

Click **Logs** to open the live log panel for the Gateway container.

2.4.2.2 Starting and Stopping

- Click **Install & start** if the Gateway has not been installed yet.
- Click **Start** if the Gateway is stopped.
- Click **Stop** (requires confirmation) to stop the running Gateway.

When the Gateway is started for the first time, Feniks also writes an MCP configuration entry to your selected harness automatically.

2.4.2.3 MCP Providers

When the Gateway is running, the MCP Providers section lists all available MCP server integrations with their status:

Status	Meaning
healthy	The provider is running and connected.
starting	The provider is initialising.
unhealthy	The provider failed to start or lost its connection.
not connected	The provider is not running.

Table 6: Feniks MCP provider status

Each provider row also shows:

- How many AI clients are currently connected to it.
- A switch to toggle the MCP server on and off.
- A configuration button (cogwheel) to set environment variables that the MCP server is dependent on.

Click the **refresh icon** to manually refresh the provider list. It also auto-refreshes every 5 seconds while the Gateway is running.

Click the **Install Additional MCPs** to manually configure or install/reinstall MCP servers from the same list presented in the installation, see section 2.2.1.3.

2.5 Skills

The Skills tab lets you browse, install, and manage AI skills across your projects, so called workspaces. Skills are reusable instruction sets, templates, and configurations that extend what the agent can do in your specific context.

When navigating to the Skills tab in the top menu. The application will be shown as illustrated in the figure below:

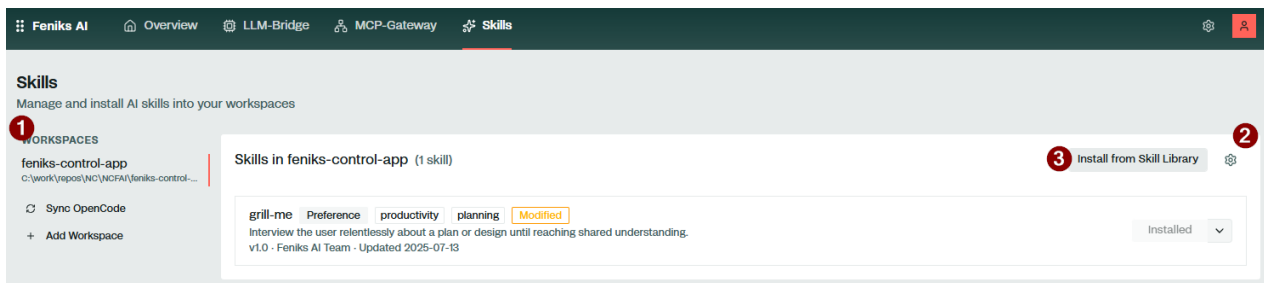


Figure 14: Feniks AI Skills

Point number in figure	Related subsection
1	2.5.1
2	2.5.2
3	2.5.3

Table 7: Feniks AI skills legend

2.5.1 Workspaces

Skills are installed per workspace. A workspace represents a project directory where OpenCode For Feniks operates. Before installing skills, you need to add at least one workspace.

The left panel of the Skills page lists your configured workspaces:

- **Select a workspace** to view its installed skills, see section 2.5.2.
- **Sync from OpenCode** — automatically import workspaces from OpenCode's project list.
- **Add workspace** — manually add a new workspace by specifying its name and path on your computer.

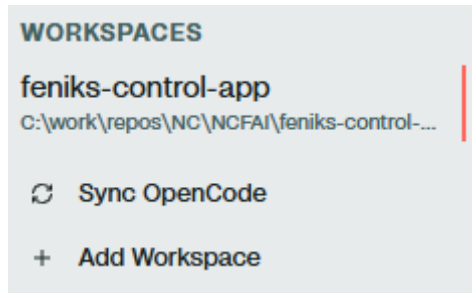


Figure 15: Feniks AI skills workspaces

If no workspaces are configured, you will see a prompt to add one or sync from OpenCode For Feniks.

2.5.2 Installed Skills

The right panel shows skills currently installed in the selected workspace. Each installed skill card shows:

- **Skill name and description** — Identifies the skill and its purpose.
- **Version and maintainer** — Tracks which version is installed.
- **Content hash status** — Indicates whether the on-disk content matches the catalogue hash. A mismatch may indicate the skill was modified locally or is corrupted.

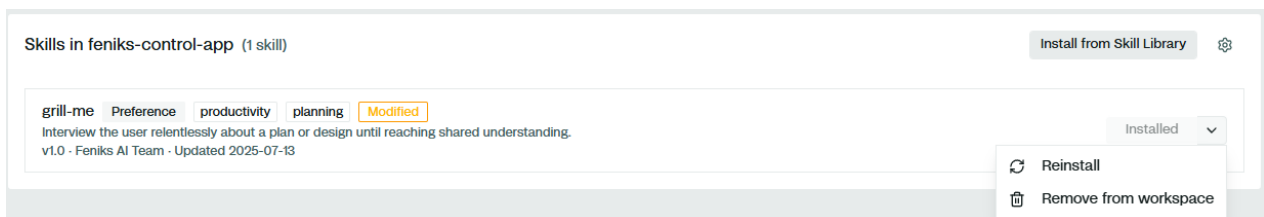


Figure 16: Feniks AI installed skills

For each installed skill, you can perform the following actions:

1. **Reinstall:** Re-download and replace the skill with the latest version from the catalogue. Use this if the content hash does not match.
2. **Remove:** Delete the skill from the workspace. This removes the skill's directory from your project.

Click the **cogwheel icon** to update the selected workspace such as the name or path of the workspace, or removing the workspace from Feniks AI.

2.5.3 Skill Library

Click **Install from Skill Library** to browse the available skills catalogue. The Skill Library provides:

- **Search bar** — Filter skills by name or description.
- **Category filters** — Narrow results by skill category (e.g., "Testing", "Documentation", "Architecture"). Categories are sorted by popularity, with a "+N more" toggle to reveal additional categories.
- **Scope filters** — Filter by scope **Global** Skills available to all projects, and **Platform** Skills scoped to a specific platform (e.g., Pulse, Ampio Lens).
- **Pagination** — Navigate through results with configurable page size (5, 10, or 20 per page).

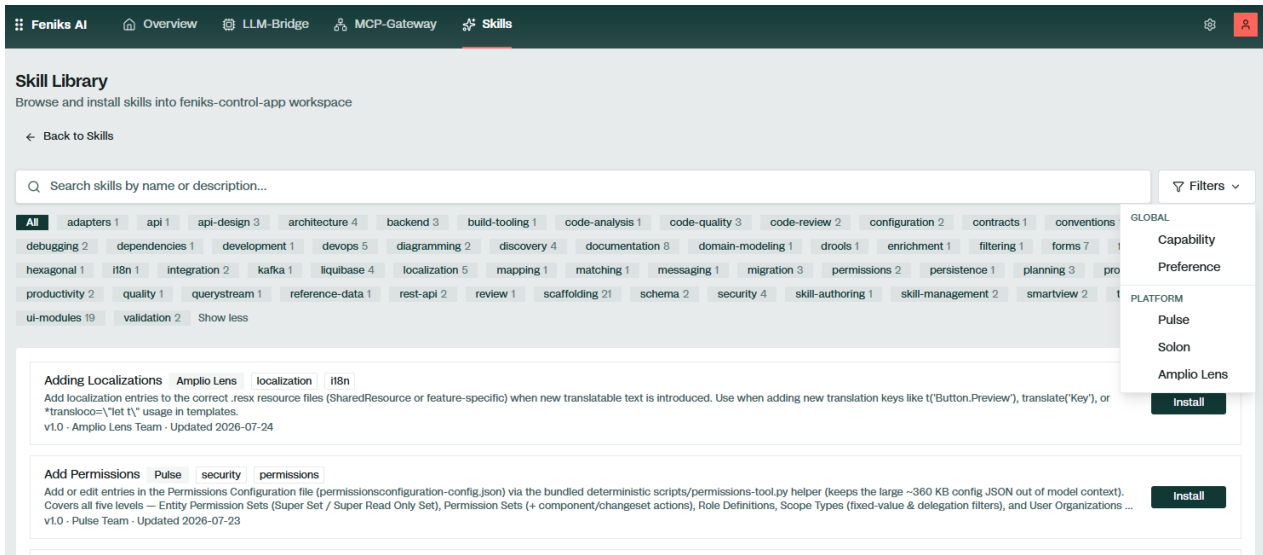


Figure 17: Feniks AI skill library

Each skill card displays the skill name, description, version, maintainer, last updated date, scope badges, and category tags. Click a skill card to open the Skill Detail dialog, see section 2.5.3.1.

2.5.3.1 Skill Detail

Click any skill card in the Skill Library to open the Skill Detail dialog. It shows the following:

- **Skill metadata** — Name, description, version, maintainer, categories, scope, and last updated date.
- **File tree** — A browsable list of files included in the skill, including the primary `SKILL.md` file.
- **File preview** — Click any file in the tree to view its contents.

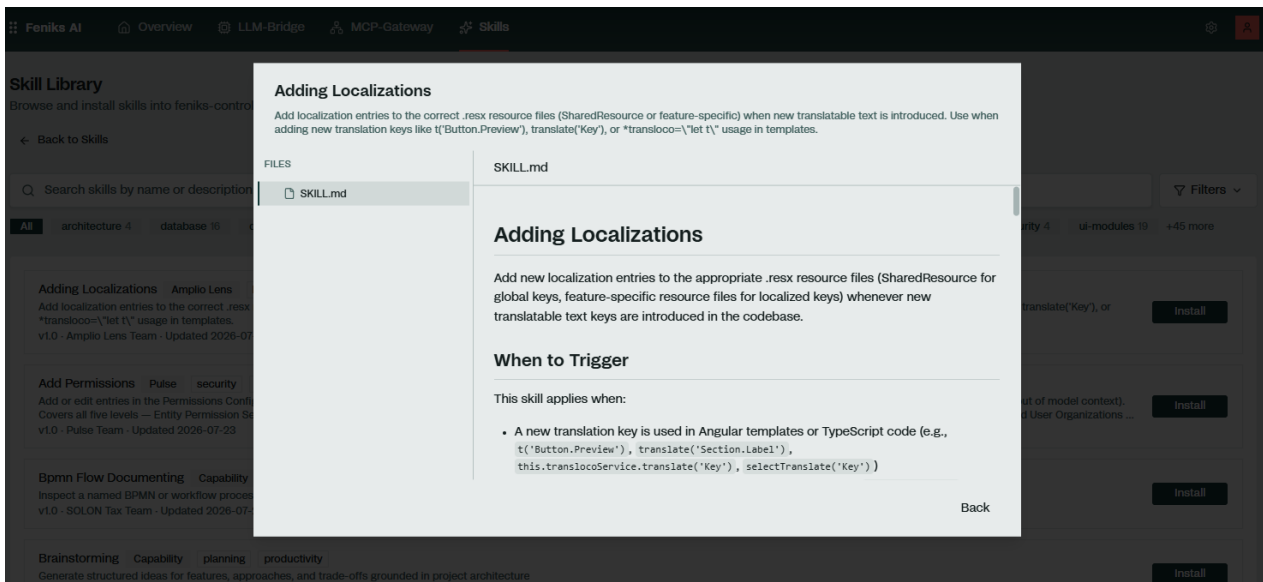


Figure 18: Feniks AI skill detail

From the Skill Library, you can install a skill directly into the current workspace or open the multi-workspace install dialog to select multiple target workspaces.

2.6 Settings

When navigating to the **Settings** (cogwheel icon ) in the top-right menu. The application will be shown as illustrated in the figure below:

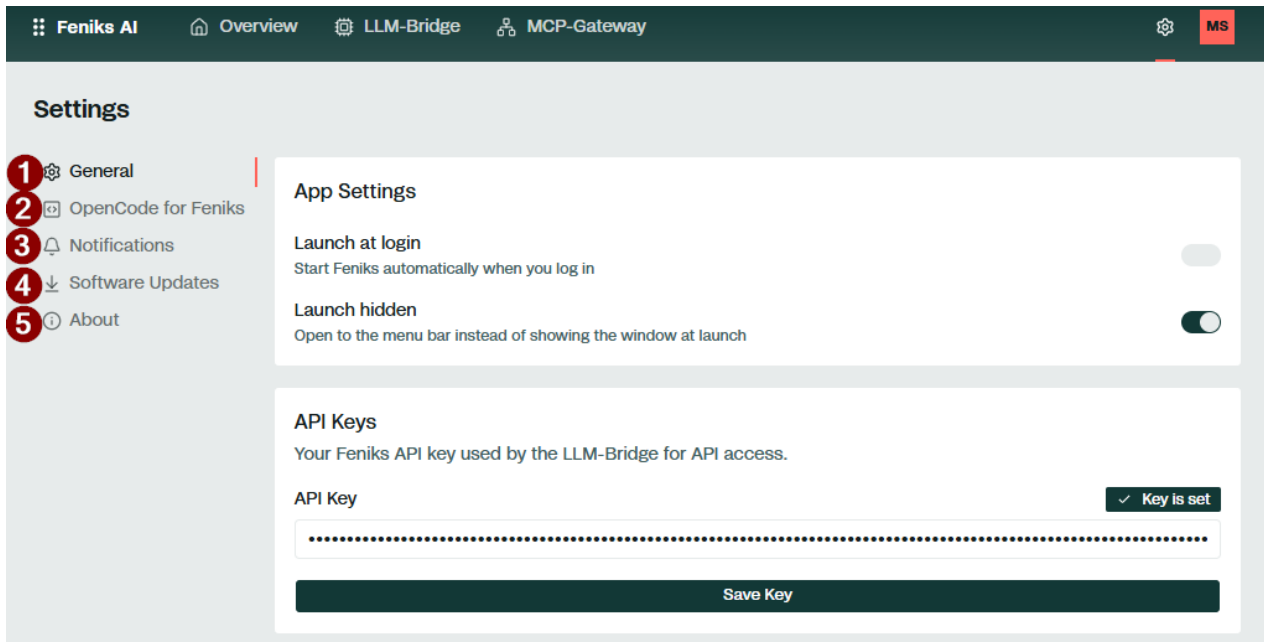


Figure 19: Feniks AI Settings

Point number in figure	Related subsection
1	2.6.1
2	2.6.2
3	2.6.3
4	2.6.4
5	2.6.5

Table 8: Feniks AI settings legend

2.6.1 General

The **General** tab exposes general application settings affecting the whole applications.

2.6.1.1 App Settings

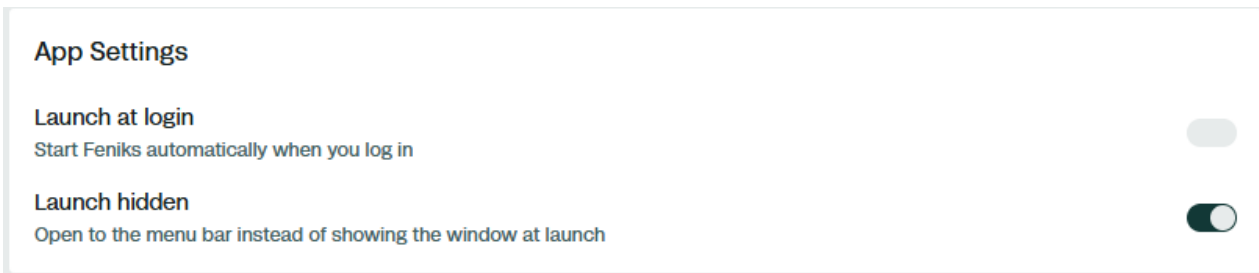


Figure 20: Feniks AI app settings

Setting	Description
Launch at login*	Feniks starts automatically when you log into Windows.
Launch hidden	When enabled, Feniks opens to the system tray rather than showing the window on startup.

Table 9: Feniks AI application settings

*This is enabled when installing. If you have also set your Rancher to automatically start at login, Feniks AI will open first, which most likely will cause an error. This can be avoided by disabling this feature in Feniks AI and start manually.

2.6.1.2 API Keys

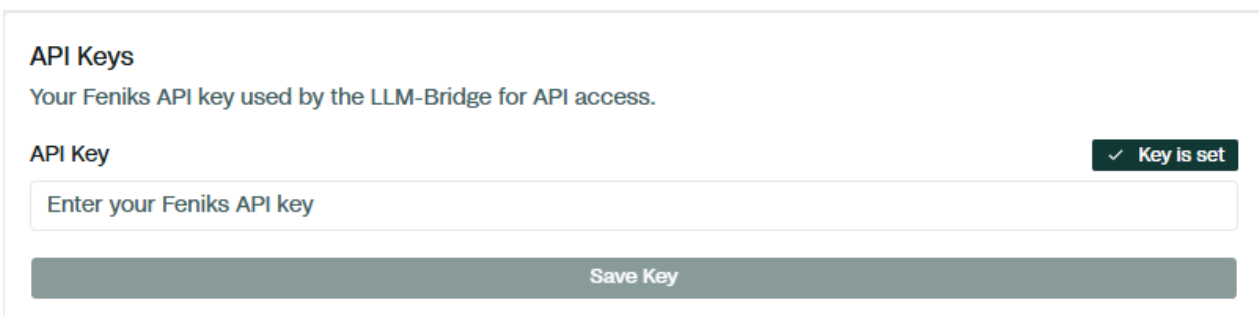


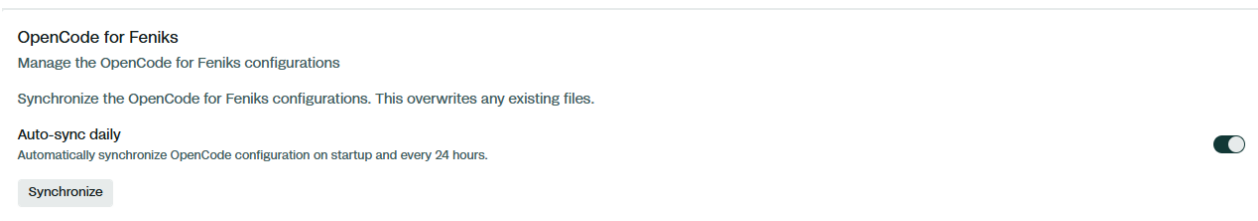
Figure 21: Feniks AI API keys settings

The **API Keys** module displays whether a Feniks API key is currently configured (shown as a **Key is set** or **No key configured** badge). To update the key:

1. Enter the new key in the **API Key** field, see section 2.2.1.2 how to obtain it.
2. Click **Save Key**.

After saving a new key, restart the Feniks Bridge container for the change to take effect (go to **Bridge** → **Stop** → **Start**).

2.6.2 OpenCode For Feniks



The **OpenCode For Feniks** tab controls the configuration of the installed AI harness that comes with Feniks AI.

The Auto-sync daily setting controls if the latest configuration from the central registry should be downloaded at application startup and every 24h since startup.

The synchronize button will immediately download the latest configuration from the central registry.

For both cases any local changes to the configuration file will be overwritten.

2.6.3 Notifications

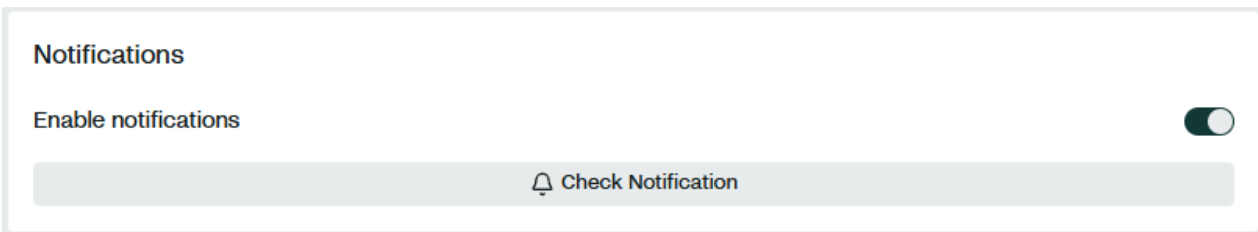


Figure 22: Feniks AI notifications settings

The **Notifications** tab controls whether Feniks sends desktop notifications (for example, when a component update is applied or a container changes state).

Setting	Description
Enable notifications	Toggle all Feniks desktop notifications on or off.
Check Notification	Sends a test notification to confirm that your operating system has granted notification permission.

Table 10: Notification settings

If you click **Check Notification** and nothing appears, your OS may have blocked notification permission for Feniks. On Windows, open **Settings** → **System** → **Notifications** and enable notifications for Feniks AI.

2.6.4 Software Updates

Software Updates Feniks v1.0.0 Up to date - just now		Check for updates
Feniks AI	v1.0.0	Up to date
OpenCode for Feniks	v1.17.8-2.2	Up to date ▼
feniks-bridge	v1.0.1	Up to date
feniks-mcp-gateway	v1.0.0	Up to date
mcp-atlassian	v1.0.0	Up to date
mcp-sonarqube	v1.0.0	Up to date
mcp-toolkit	v1.0.0	Up to date

Figure 23: Feniks AI Software updates settings

The **Software Updates** tab enables user to check for and apply updates to Feniks AI and its component container images.

Checking for updates:

1. Click **Check for updates**.
2. Feniks AI compares each component's current image tag against the registry.
3. One of the following badges/buttons are shown for each component:
 - a. **Update available** badge and **Apply** button – Newer version is discovered in the registry compared to already installed component.
 - b. **Up to date** – The latest version of the component is already installed.
 - c. **Updated** – The latest version of the component has been applied.

Applying updates:

1. After a check finds updates, an **Apply all updates** button appears.
2. Click it. Feniks pulls the new images and automatically restarts any running containers that were updated.
3. A live pull log is shown during the download.

The current Feniks AI app version is shown at the top of this tab. The app itself is updated automatically at startup via the built-in update check, see section 2.2.1.1.

2.6.5 About

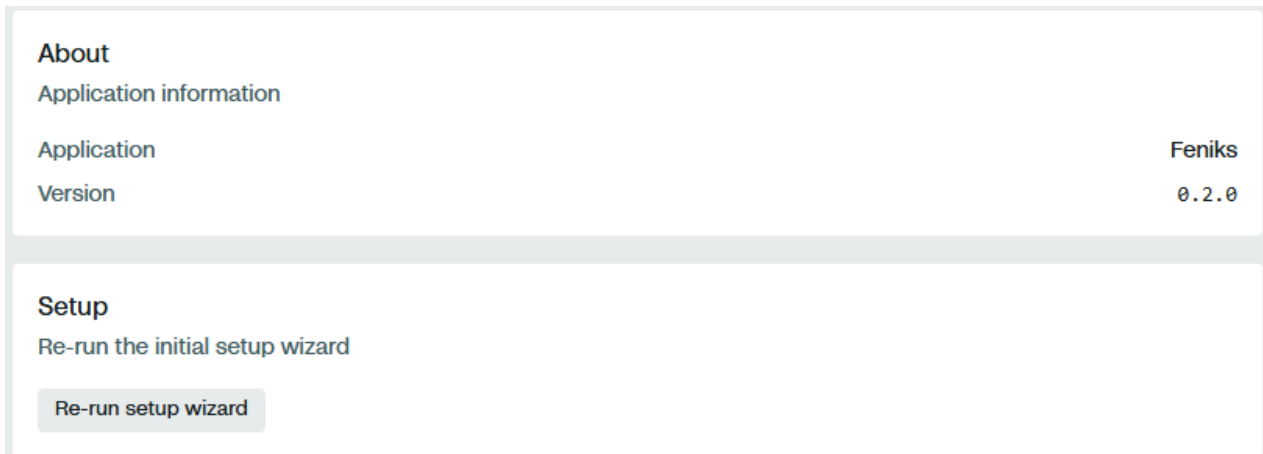


Figure 24: Feniks AI about settings

The **About** tab displays the current application version and build information. A user can also re-initialize the same first-time setup flow started in section 0 by clicking the button **Re-run setup wizard**.

2.7 User Menu

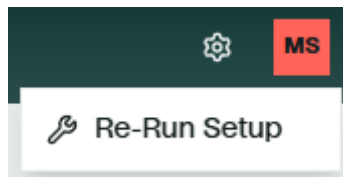


Figure 25: Feniks AI user menu

Click the icon in the very top-right corner of the app to access the user menu.

Action	Description
Run Setup	Re-runs the first-time setup flow, see 0. Use this to re-install components.

Table 11: Feniks AI user menu actions

2.8 Troubleshooting

NOTE: This section has been moved to the [FAQ](#) section in the NCFAI Toolkit.

3 OpenCode For Feniks Agentic AI Assistant

This chapter covers how to use OpenCode For Feniks effectively in your project work. Before diving in, keep the following in mind:

- **Start with the right interface.** Choose the Desktop App if you prefer visual workflow, or the Terminal Interface if you are comfortable on the command line. There is no wrong choice — sessions are shared between both.

- **Use agents intentionally.** Switch to the Plan agent when exploring or designing and reserve the Build agent for active implementation. Starting a new session when switching tasks keeps context clean and the model focused.
- **AGENTS.md is the foundation.** Every project should have a well-maintained AGENTS.md file committed to Git. Never leave the auto-generated file unedited — research shows it performs worse than having no file at all if left unreviewed.
- **Skills and MCP servers multiply your productivity.** Invest time in setting up project-specific skills and local MCP servers such as IntelliJ or PostgreSQL. These dramatically extend what the agent can do in your specific context.
- **Only local MCP servers are permitted.** Connections to external services such as Jira, Confluence, or any SaaS platform are explicitly prohibited at Netcompany due to prompt injection risk.
- **You are always responsible for the output.** Agents can produce plausible but incorrect code. Review everything, use Git-backed undo when needed, and treat the agent as a capable colleague that still requires oversight.
- **Choose the right model for the task.** Not every task requires the most powerful model. Simple, well-defined tasks should be handled with a cheaper, faster model such as Haiku. Reserve Sonnet for larger, more complex problems — particularly when working with extensive skills or multi-step agentic workflows where reasoning quality matters.

This section provides step-by-step instructions for configuring and using OpenCode For Feniks. It covers the Desktop App and Terminal Interface, project configuration, and day-to-day usage.

For best practices, methodology, and project-level configuration guidance, refer to [\[C0200 – Agentic AI Guidelines\]](#) for inspiration.

3.1 Working with OpenCode For Feniks

OpenCode For Feniks is a revamped version of OpenCode adopted by Netcompany. OpenCode at its heart is an open-source, terminal-based agentic coding assistant. Unlike IDE-integrated tools, OpenCode For Feniks runs in any terminal and is not tied to a specific editor or graphical user interface. It supports multiple LLM providers through a unified configuration file and can be extended with MCP servers for additional tool access.

3.1.1 The OpenCode For Feniks Interface

OpenCode For Feniks is available in two forms: a Desktop App with a graphical interface and a Terminal Interface (TUI) for command-line use. Both are installed via Feniks AI and share the same underlying engine, configuration, agents, skills, and sessions. For details on installation please refer to the [\[C0200 – Agentic AI Guidelines\]](#).

3.1.1.1 Choosing an Interface

There are two main choices in interface, a TUI (terminal interface) and a GUI (Graphical User interface). The table below shows the differences between the two.

Desktop App (GUI)	Terminal Interface (TUI)
Open from Start Menu or taskbar	Run OpenCode For Feniks CLI in a terminal
Best for developers who prefer a visual interface, those new to agentic AI, and longer planning sessions	Best for developers comfortable with the terminal, fast keyboard-driven workflows, and scripting-heavy tasks

Session management via sidebar with clickable session list	Session management via slash commands (/new, /sessions)
Agent switching via click in the UI	Agent switching via Tab key
File references via click to browse or type @	File references via @ with fuzzy search
Model selection via dropdown	Model selection via /models or F2
Undo/redo via interface buttons	Undo/redo via /undo and /redo commands
Image support via drag and drop	Image support via drag and drop (terminal dependent)

Table 12: OpenCode For Feniks GUI vs TUI

There is no wrong choice. Sessions are shared between interfaces, so you can start work in one and continue in the other.

3.1.1.2 Choosing a Model

You can change the model at any time during a session — in the Desktop App via the model selector dropdown, or in the TUI via /models or F2. Model selection applies to the current session only and does not affect other open sessions.

Important: Refer to the [C0200 – Agentic AI Guidelines] for details on how to select the right model for the right task.

3.1.1.3 Desktop App (GUI)

After installation, open the Desktop App from the Start Menu or pin it to the taskbar. On first launch it will detect your Feniks API key and connect automatically. You need to open a project directory to start working.

3.1.1.3.1 Interface Overview

The Desktop App interface consists of the following elements:

- **Session sidebar (left):** Lists all your sessions. Click to switch between them or create a new session.
- **Chat pane (centre):** Shows the conversation with the agent, including tool calls, file edits, and responses.
- **Input area (bottom):** Where you type prompts. Supports @ file references, image drag-and-drop, and slash commands.
- **Agent selector (top or status bar):** Shows the currently active agent. Click to switch.
- **Model selector:** Choose which model to use for the current session.

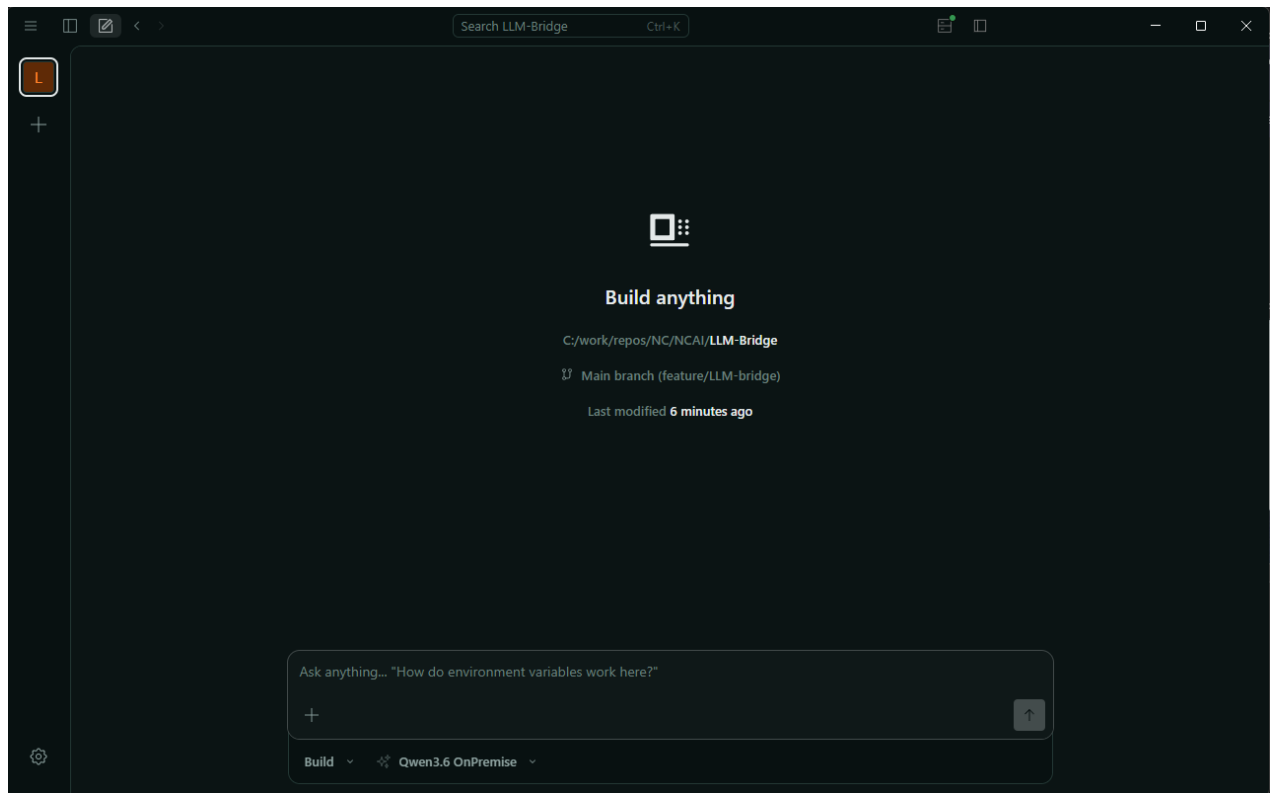


Figure 26: OpenCode For Feniks desktop app GUI

3.1.1.3.2 Working with Sessions

Create a new session by clicking the new session button in the sidebar. Each session is tied to a project directory. To clear context and start fresh on a new task, create a new session rather than continuing in a long-running one. You can also type `/compact` to summarise the current session and reclaim context space.

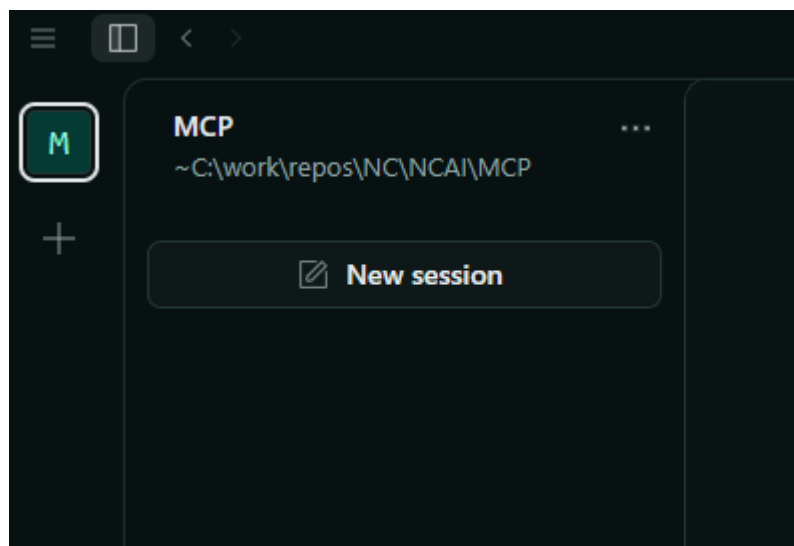


Figure 27: OpenCode For Feniks GUI sessions

3.1.1.3.3 File References and Images

Type @ in the input area to search for and reference project files. The file content is added to the conversation automatically. You can also drag and drop images directly into the chat to add visual context to your prompts.

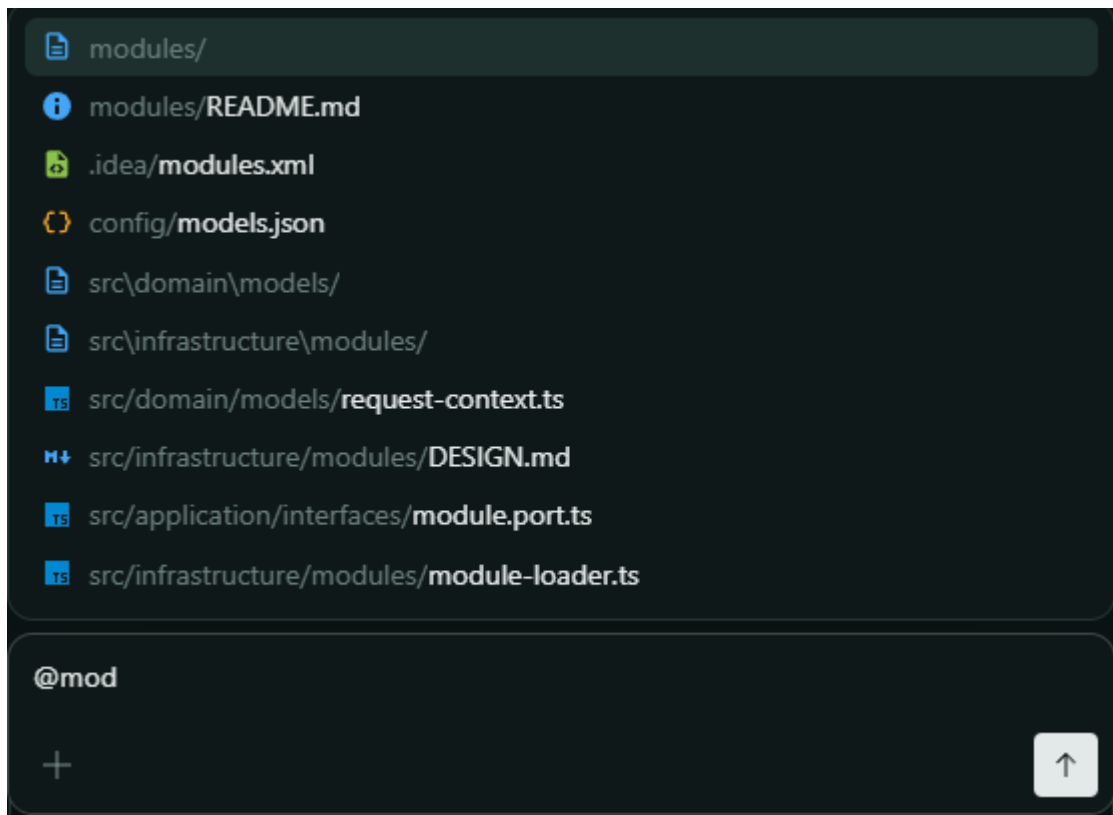


Figure 28: OpenCode For Feniks GUI file references

3.1.1.3.4 Undo and Redo

The Desktop App provides undo/redo through the interface. You can also type /undo or /redo. Both the messages and any file changes the agent made are reverted or restored. Your project must be a Git repository for this to work.

3.1.1.4 Terminal Interface (TUI)

Launch the TUI by running OpenCode For Feniks CLI in the directory containing your project files.



Figure 29: OpenCode For Feniks TUI

3.1.1.4.1 Interface Layout

The TUI fills the terminal window with a chat pane showing the conversation, an input field at the bottom for typing prompts, and a status bar displaying the active agent, model, and session information.

3.1.1.4.2 File References

Use `@` in your prompt to reference files. This performs a fuzzy file search in the current project directory and adds the file content to the conversation. Example:

```
How is auth handled in @packages/functions/src/api/index.ts?
```

3.1.1.4.3 Running Shell Commands

Start a message with `!` to run a shell command directly. The output is added to the conversation as context. Example:

```
!ls -la
```

3.1.1.4.4 Slash Commands

Type `/` followed by a command name to execute actions. These commands work in both the TUI and the Desktop App.

Command	Keybind	Description
<code>/new</code>	Ctrl+X N	Start a new session. Use when switching to a new task.
<code>/init</code>	Ctrl+X I	Generate or update the project AGENTS.md file.
<code>/compact</code>	Ctrl+X C	Summarise the current session to free up context space.
<code>/undo</code>	Ctrl+X U	Undo the last message and revert any file changes.
<code>/redo</code>	Ctrl+X R	Redo a previously undone message and restore file changes.
<code>/models</code>	Ctrl+X M	List and select available models.
<code>/sessions</code>	Ctrl+X L	List and switch between previous sessions.
<code>/editor</code>	Ctrl+X E	Open an external editor for composing longer prompts.
<code>/details</code>	Ctrl+X D	Toggle tool execution details in the conversation.
<code>/thinking</code>		Toggle visibility of the model reasoning blocks.
<code>/help</code>	Ctrl+X H	Show the help dialog with all available commands.
<code>/share</code>	Ctrl+X S	Share the current session via a link.
<code>/export</code>	Ctrl+X X	Export the conversation to Markdown.
<code>/exit</code>	Ctrl+X Q	Exit OpenCode For Feniks. Also: Ctrl+C or Ctrl+D.

Table 13: OpenCode For Feniks commands

3.1.1.4.5 Key Navigation

The TUI uses Ctrl+X as the leader key. Most keybinds require you to first press Ctrl+X and then the shortcut key.

Action	Keybind
Switch between primary agents (Build/Plan)	Tab
Cycle agents in reverse	Shift+Tab
Open command palette	Ctrl+P
Interrupt a running response	Escape
Cycle through model variants (thinking modes)	Ctrl+T
Cycle recent models	F2
New line in input (without sending)	Shift+Enter
Navigate to child session (subagent)	Leader + Down
Cycle child sessions	Right / Left arrows
Return to parent session	Up arrow

Table 14: OpenCode For Feniks keybindings

3.1.1.4.6 Session Management

Each session accumulates context as the conversation grows. Over time, a long session can cause the model to become slower and less precise because it must process a larger amount of prior information.

- Start a fresh session with `/new` when moving to a new task or topic.
- Use `/compact` to summarise history and reclaim context space if a session grows large.
- Use `/sessions` to browse and switch between saved sessions.

3.1.1.4.7 Undo and Redo

OpenCode For Feniks integrates with Git to provide undo/redo for both messages and file changes. Running `/undo` removes the last user message, all subsequent responses, and reverts any file changes the agent made. Running `/redo` restores them. Your project must be a Git repository for this to work.

3.1.2 Working with Agents

OpenCode For Feniks uses the term "agent" to refer to a configured AI persona with its own system prompt, model, tool permissions, and behavioural constraints. Multiple agents coexist within a single installation and serve different roles.

3.1.2.1 Built-in Agents

OpenCode For Feniks uses agents to represent different AI personas, each with its own system prompt, model, tool permissions, and behavioural constraints. Two agents are built in:

- **Build** is the default agent with full tool access (file read/write, bash execution, web fetch). Use it for implementation tasks: writing code, running tests, applying changes.
- **Plan** is a read-only agent. It can analyse code and reason about the codebase but cannot make changes. Use it to propose approaches, review designs, or explore code without risk of unintended modifications.

Subagents are specialised assistants that the active agent can delegate work to in an isolated session, keeping the primary conversation context clean. OpenCode For Feniks ships with General (full access) and Explore (read-only) subagents.

Switch between primary agents by pressing Tab in the TUI, or by clicking the agent selector in the Desktop App.

3.1.2.2 Using Subagents

Subagents are specialised assistants that the active agent can delegate work to in an isolated session. The isolation keeps the primary conversation focused because the subagent's work does not accumulate in the main context window.

OpenCode For Feniks ships with two built-in subagents:

- **General:** A general-purpose subagent with full tool access for researching complex questions and executing multi-step tasks.
- **Explore:** A fast, read-only subagent for navigating and understanding codebases. Cannot modify files.

3.1.2.2.1 Invoking Subagents

The active agent decides when to invoke a subagent automatically based on the task. You can also invoke a subagent explicitly using the @ prefix:

```
@explore find all files related to authentication and describe the overall structure
```

To navigate between parent and child sessions in the TUI, use Leader + Down to enter a child session, Right/Left arrows to cycle between child sessions, and Up arrow to return to the parent.

3.1.2.3 Creating a Custom Agent

Custom agents are defined as standalone markdown files placed in `.opencode/agents/` (project-level) or `~/config/opencode/agents/` (global). The markdown filename becomes the agent name.

Custom agents are worth creating when a recurring workflow benefits from a distinct system prompt, tool restriction, or model choice. Avoid creating agents for one-off variations that can be handled with a good prompt to the Build agent

3.1.2.3.1 Agent File Format

A markdown-based agent definition has YAML frontmatter for configuration and a markdown body that becomes the system prompt:

```
---
description: "Reviews code changes for correctness, style, and security."
model: claude-4-sonnet
temperature: 0
mode: primary
permission:
  bash:
    "git diff*": allow
    "*": deny
```



```

    edit: deny
tools:
  skill: false
---
```

You are a code reviewer on a Netcompany project.

Focus on:

- Correctness and edge cases
- Adherence to conventions defined in AGENTS.md
- Security issues (secrets, injection, auth)

Provide constructive feedback without making direct changes.

3.1.2.3.2 Configuration Fields

OpenCode For Feniks can scaffold a new agent. Run the agent creation command and it will ask where to save the agent, what it should do, generate a system prompt, let you select tools, and create the markdown file. Review and trim the output before use.

Field	Description
model	Which LLM to use. Run OpenCode For Feniks models to list available models.
temperature	Controls randomness. 0 = deterministic, higher = more creative.
mode	primary (Tab rotation), subagent (@mention only), or all (both).
permission	Controls bash, edit, and webfetch tool access. Supports glob patterns.
tools	Enables/disables tool categories. E.g. skill: false, bash: false.
max_iterations	Caps agentic loops before the agent must respond with text only.

Table 15: OpenCode For Feniks custom agent fields

3.2 Using OpenCode For Feniks in your project

3.2.1 Adding Custom Instruction Files

OpenCode For Feniks allows additional instruction files to be loaded alongside AGENTS.md through the instructions field in a project-level opencode.json:

```

{
  "$schema": "https://opencode.ai/config.json",
  "instructions": [
    "CONTRIBUTING.md",
    "docs/guidelines.md",
    ".cursor/rules/*.md"
  ]
}
```

Glob patterns are supported. Remote URLs can also be specified (fetched with a 5-second timeout). All instruction files are combined with AGENTS.md content.

3.2.2 Embedding File-Read Directives

For large projects where the full set of rules would exceed a reasonable AGENTS.md size, you can instruct the agent to read external files on demand:

```
## External references
CRITICAL: When you encounter a file reference (e.g., @rules/general.md),
use your Read tool to load it on a need-to-know basis.

Read the following file immediately as it is relevant to all workflows:
@rules/general-guidelines.md
```

3.2.3 Setting Up MCP Servers

MCP (Model Context Protocol) servers extend what the agent can see and do beyond the files in your repository. They expose capabilities such as database queries, code intelligence from your IDE, or cross-repository search through a standardised protocol.

IMPORTANT: Only MCP servers operating exclusively on local data are recommended, except for MCPs available in the MCP-Gateway install list. MCP servers connecting to remote services (Jira, Confluence, SharePoint, Toolkit, or any external SaaS) are per default allowed on Netcompany domain and only allowed with client hosted servers with their approval. Other MCP server configurations are not recommended due to prompt injection risk. The Trust Centre mitigates, what outstanding risk remains, and under what conditions external MCPs can be used.

The rationale: when an agent pulls content from a remote service, that content enters the agent's context window and can contain adversarial instructions that hijack the agent's behaviour. Restricting MCP to local-data servers eliminates this attack surface.

3.2.3.1 Adding a MCP Server

OpenCode is capable of handling multiple configuration sources at different levels by merging them into one. Since Feniks AI governs the global configuration one can construct an `opencode.json` file at project root with valid syntax following: <https://opencode.ai/config.json>. For example:

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "intellij": {
      "enabled": true,
      "type": "remote",
      "url": "http://127.0.0.1:64342/sse",
      "headers": {
        "IJ_MCP_SERVER_PROJECT_PATH": "C:/work/repos/NC/NCFAI/feniks-control-app"
      }
    }
  }
}
```

Looking at the active MCPs for the project shows that it has been loaded in successfully (the MCP will not show up if it has not been configured correctly):

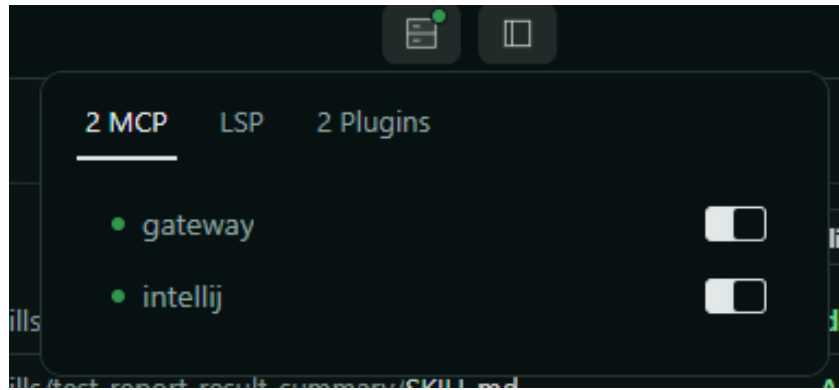


Figure 30: Verify MCP server in OpenCode For Feniks

One can also run the command at project root to verify the merged configuration is the expected one:

opencode debug config

In this case showing both the Feniks MCP gateway and my project specific IntelliJ MCP.



Figure 31: Verify MCP server in command-line

3.2.3.2 Atlassian Jira and Confluence MCP

Please refer to [C0200 – Jira Confluence MCP Setup].

3.2.3.3 SonarQube MCP

The SonarQube MCP server exposes code quality and security data features (analysis, issues, quality gates, security hotspots, coverage, projects, dependency risks, context augmentation) from remotely hosted server over a HTTP connection.

1. Obtain Netcompany SonarQube token by creating a new personal access token here: <https://sonarqube.netcompany.com/account/security>
2. Save the token as a value to `SONARQUBE_TOKEN` configuration, see section 2.4.2.3.
3. Restart the MCP Gateway in Feniks AI, see section 2.4.2.2.

Please note that the MCP server is pre-configured with Netcompany's hosted SonarQube server by default when installing Feniks AI. To add a customer's hosted server, one must add a new MCP server by adding another provider to the configuration, see section 3.2.3.1.

You can also point the SonarQube MCP at a customer's hosted server by setting SONARQUBE_URL in the Configure dialog (MCP-Gateway tab, select mcp-sonarqube, Configure), save, and restart the MCP Gateway. This replaces the need to add a separate provider. The SONARQUBE_TOKEN must be valid for that instance.

3.2.3.4 IntelliJ MCP

The IntelliJ MCP server exposes code intelligence features (go to definition, find usages, rename refactoring) from your IDE to OpenCode For Feniks over a local SSE connection.

1. Enable the MCP server in IntelliJ under Settings and note the port number.

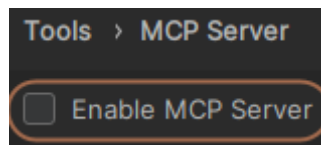


Figure 32: IntelliJ MCP Server needs to be explicitly enabled

2. Add the following to your project-level opencode.json:

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "intellij": {
      "type": "remote",
      "url": "http://127.0.0.1:64342/sse",
      "enabled": true
    }
  }
}
```

Consider adding the following to your AGENTS.md to guide the agent on when to use IntelliJ tools:

```
## Code Intelligence
Prefer IntelliJ MCP tools for code navigation and refactoring:
- Use intellij_get_symbol_info instead of reading files for type info
- Use intellij_find_usages_in_project instead of grep for finding references
- Use intellij_rename_refactoring for any rename operations
- Always provide projectPath={rootPath}\src parameter

### IntelliJ MCP: What NOT to use it for
> NEVER use IntelliJ MCP tools as a substitute for shell/terminal operations.
> The IntelliJ terminal tool is strictly for IDE-specific actions.
> Do NOT use it to run Python scripts, Gradle builds, or any general
> shell command that the bash tool can handle.
```

3.2.4 PostgreSQL MCP Pro

Connects the agent to a **local** PostgreSQL development database. Only use with local dev databases, never with shared or production environments.

```
{
  "mcp": {
    "postgres": {
```

```

    "type": "local",
    "command": ["docker", "run", "-i", "--rm",
        "-e", "DATABASE_URI",
        "crystalldba/postgres-mcp",
        "--access-mode=restricted"
    ],
    "environment": {
        "DATABASE_URI": "postgresql://username:password@localhost:5432/db_name"
    }
}
}
}

```

Always use `--access-mode=restricted` for read-only transactions. Use `--access-mode=unrestricted` only for local development where full read/write is acceptable.

3.2.4.1 Sourcebot

Sourcebot indexes your locally checked-out Git repositories and makes them searchable by the agent. Use the local file indexing approach, not remote repository connections.

```

{
  "mcp": {
    "sourcebot": {
      "type": "remote",
      "url": "http://localhost:3000/api/mcp",
      "oauth": false,
      "headers": {
        "Authorization": "Bearer {replace with your API key}"
      }
    }
  }
}

```

Consider adding the following to your `AGENTS.md`:

```

## Sourcebot MCP usage
- When using the search_code tool from Sourcebot, always set maxTokens to 50000
  unless the user explicitly asks for a summary.
- Never truncate results without informing the user and offering to search with
  a more specific query.
- Do not use ask_codebase method.

```

3.2.5 Toolkit MCP

Please refer to [\[C0200 – Toolkit MCP Setup\]](#)

3.2.6 Configuring File Exclusions

OpenCode For Feniks' search tools respect `.gitignore` patterns by default, but `.gitignore` is not a hard security boundary. The read tool can still access files by absolute path. The recommended approach is to use three layers together:

1. `.gitignore` for file-level exclusion from search and discovery.

2. AGENTS.md for explicit scope directives that tell the agent which directories to work in and which to avoid.
3. Prompt-level instructions for sensitive tasks as a redundant safety layer.

3.2.6.1 Recommended .gitignore Entries

```
# Heavy directories that waste tokens
node_modules/
dist/
build/

# Secrets and credentials
.env
*.key
*.pem

# Logs
*.log
```

3.2.6.2 Limitations

The read tool can still access files by absolute path even if they are listed in .gitignore. For files that must never be read, add explicit directives in AGENTS.md:

```
## Project Structure
- src/ : main application code, agent should work here
- scripts/ : build/deploy scripts, agent may read but not modify
- node_modules/ : DO NOT read or analyze, ever
- .env : DO NOT read, contains secrets
- dist/ : generated output, ignore entirely
```

3.2.6.3 Prompt-Level Instructions

For sensitive tasks, add a direct instruction in the prompt itself as a redundant layer:

```
Do not read or modify anything in node_modules/, dist/, or .env.
```

The recommended approach is to use all three layers together: .gitignore for search exclusion, AGENTS.md for scope directives, and prompt-level instructions for sensitive tasks.

3.3 Getting Help

OpenCode For Feniks provides several built-in tools to help you understand what the agent is doing and troubleshoot issues.

3.3.1 Debugging Tools

Command	What it does
/help	Shows the help dialog with all available commands and keybinds.
/thinking	Toggles visibility of the model's internal reasoning blocks. Useful for understanding why the agent made a particular decision.

/details	Toggles tool execution details in the conversation. Shows exactly which tools were called and what they returned.
/export	Exports the conversation to Markdown. Useful for review, documentation, or attaching to support requests.

Table 16: OpenCode For Feniks debugging tool commands

3.3.2 Resources

- **Netcompany agentic AI e-learning modules:** Training materials covering agentic AI concepts and hands-on exercises.
- **Feniks AI support:** For installation issues, OpenCode For Feniks update failures, API key issues, token limits, or model availability questions.
- **Your project's AGENTS.md and SKILL.md:** Project-specific conventions, commands, and workflows. Always check these first for project-specific guidance.

3.4 Known pitfalls and problems

This chapter covers known issues and pitfalls. It should be used to get an overview of common issues and users of OpenCode For Feniks should be aware of these issues.

3.4.1 Looping behaviour in plan mode

In plan mode, OpenCode For Feniks blocks all write and execute operations at the permission layer. The agent itself is not informed that it is running in plan mode, so when it reaches the point of applying changes it attempts a write, receives a denial, acknowledges the issue in its output, and then immediately retries the same operation. This pattern can continue until the entire token budget is consumed with no productive work done.

How to recognize it

- The agent produces repeated tool calls (often bash echo, Write, or Edit) that are all denied.
- Between the denied calls, the agent generates self-correcting text such as "I realise I keep calling bash" or "I will stop using this tool," followed immediately by another attempt at the same blocked operation.
- The agent will not recover from this on its own.

What to do

- Stop the agent as soon as you see the pattern. Do not wait for it to self-correct.
- Compact the current session, or start a new one.
- If you start a new session, include an explicit instruction describing what you want (for example, "present a plan only, do not apply changes").

How to avoid it

- Watch the agent's output during long-running tasks. Catching a loop in the first few iterations costs almost nothing; letting it run unattended can exhaust your daily token budget.
- When you use plan mode intentionally, state up front that the agent should only plan and not attempt to apply changes.

3.4.2 Burning through daily token allowance

It is possible to exhaust a full day's token budget in a single morning. This typically happens when the agent is left to run on tasks where it cannot make progress, or when sessions are set up in a way that makes the agent work harder than it needs to. The looping behaviour described in 3.4.1 is one common cause. The patterns below are the others to watch for.

Common patterns that waste tokens

- Running long tasks unattended. If the agent goes off track early, every token after that point is wasted. Catching drift in the first few steps is cheap; catching it after an hour is not.
- Starting with a vague prompt. The agent will spend a large amount of context exploring the codebase trying to work out what you want. Being specific up front about scope, files, and the desired outcome saves significant tokens.
- Skipping plan mode for non-trivial changes. If the agent applies changes you did not want, it then has to undo and redo the work. Using plan mode first to agree on an approach is almost always cheaper than recovering from a wrong execution.
- Pushing through an incoherent session. Once a session has accumulated confused state or wrong assumptions, further prompts tend to compound the problem rather than fix it. Starting a new session with a clean prompt is usually faster.
- Asking for large-scope changes in one go. The agent performs better on focused tasks. Breaking a large refactor into smaller steps uses fewer tokens overall and gives you natural checkpoints to verify progress.
- Re-running the same prompt after a failure. If a prompt did not work the first time, re-running it rarely helps. Adjusting the approach or adding context is almost always the right move.

How to avoid it

- Think through the prompt before you start. Name the files, the scope, and the expected result.
- Use plan mode first for anything beyond a trivial change. Review the plan, then execute.
- Keep an eye on the first few tool calls of any session. If the agent has misunderstood, stop it immediately.
- When a session starts producing confused or inconsistent output, start a new one rather than trying to recover.
- Break large tasks into checkpoints. It is easier to correct course after one step than after twenty.